

System skanowania lokalnej sieci komputerowej z wykrywaniem zagrożeń i raportowaniem

Mateusz Karaszewski 337036, David Gąsiorek 331168

9 kwietnia 2026

Spis treści

1	Wprowadzenie	3
1.1	Cel projektu	3
1.2	Zakres projektu	3
1.3	Motywacja i kontekst (operator telekomunikacyjny)	3
2	Założenia systemu	4
2.0.1	Wymagania funkcjonalne	4
2.0.2	Wymagania нефункционалне	4
2.1	Ograniczenia platformy MikroTik oraz mechanizmu kontenerów	5
2.1.1	Ograniczenia sprzętowe	5
2.1.2	Ograniczenia systemowe i wdrożeniowe	5
2.1.3	Ograniczenia bezpieczeństwa	6
2.1.4	Ograniczenia wydajnościowe	6
2.1.5	Konsekwencje projektowe	6
3	Ogólna architektura systemu	6
3.1	Opis koncepcji rozwiązania	6
3.2	Schemat architektury	7
3.3	Przepływ danych w systemie	7
3.4	Model działania (lokalny vs zdalny)	8
3.4.1	Tryb lokalny	8
3.4.2	Tryb zdalny	8
3.4.3	Porównanie podejść	8
4	Komponenty systemu	9
4.1	Przegląd komponentów	9
4.2	Moduł skanowania sieci (Nmap + NSE)	9
4.2.1	Schemat działania modułu	10
4.2.2	Pseudokod algorytmu	11
4.2.3	Propozycja implementacji	11
4.3	Moduł analizy wyników	12
4.3.1	Schemat działania modułu	12
4.3.2	Pseudokod algorytmu	13
4.3.3	Propozycja implementacji	13
4.4	Moduł raportowania (email)	14
4.4.1	Schemat działania modułu	14
4.4.2	Pseudokod algorytmu	15
4.4.3	Propozycja implementacji	15

4.5	Moduł zarządzania (CLI/SSH)	16
4.5.1	Zakres funkcjonalny	16
4.5.2	Schemat działania modułu	16
4.5.3	Pseudokod algorytmu	17
4.5.4	Propozycja implementacji	17
4.6	Moduł harmonogramu (scheduler)	18
4.6.1	Zakres funkcjonalny	18
4.6.2	Schemat działania modułu	18
4.6.3	Pseudokod algorytmu	19
4.6.4	Propozycja implementacji	19
4.7	Kontener (Docker / LXC)	19
4.7.1	Zakres funkcjonalny	19
4.7.2	Schemat działania modułu	20
4.7.3	Pseudokod algorytmu	20
4.7.4	Propozycja implementacji	21
5	Interfejsy i komunikacja	21
5.1	Komunikacja między modułami	21
5.2	Interfejs wejściowy (konfiguracja)	21
5.3	Interfejs wyjściowy (raporty)	22
5.4	Integracja z routerem MikroTik	22
5.4.1	Algorytm działania	23
5.4.2	Pseudokod	23
6	Mechanizm skanowania sieci	24
6.1	Zakres skanowania	24
6.2	Wykorzystanie Nmap	24
6.3	Wykorzystanie NSE (Nmap Scripting Engine)	25
6.4	Optymalizacja skanowania	25
7	Mechanizm detekcji zagrożeń	26
7.1	Typy wykrywanych zagrożeń	26
7.2	Analiza wyników skanowania	26
7.3	Reguły detekcji	26
8	Mechanizm raportowania	27
8.1	Format raportu	27
8.2	Wysyłka email	28
8.3	Automatyzacja powiadomień	28
9	Wdrożenie systemu	29
9.1	Konteneryzacja (Docker / LXC)	29
9.2	Struktura projektu (GitHub)	29
9.3	Instalacja przez SSH (one-command)	30
9.4	Uruchomienie systemu	30
10	Podsumowanie	31
10.1	Założone cele	31
10.2	Możliwości rozwoju	31
10.3	Ograniczenia rozwiązania	31

1 Wprowadzenie

1.1 Cel projektu

Celem projektu jest zaprojektowanie i implementacja systemu umożliwiającego automatyczne skanowanie lokalnej sieci komputerowej użytkownika końcowego w celu wykrywania potencjalnych zagrożeń bezpieczeństwa. System działa bezpośrednio na urządzeniu - router MikroTik i wykorzystuje kontenery do uruchamiania narzędzi open-source, w szczególności programu Nmap.

Projekt ma na celu odwzorowanie modelu działania operatora telekomunikacyjnego, który zdalnie nadzoruje stan sieci klienta, przy jednoczesnym zachowaniu minimalnej ingerencji użytkownika końcowego. System powinien działać w sposób możliwie transparentny oraz umożliwiać automatyczne wykonywanie skanów i generowanie raportów.

Dodatkowym celem projektu jest przygotowanie rozwiązania w formie łatwej do wdrożenia, umożliwiającej instalację za pomocą nieskomplikowanego polecenia wykonywanego z poziomu SSH.

1.2 Zakres projektu

Zakres projektu obejmuje zaprojektowanie oraz implementację mechanizmu realizującego następujące funkcje:

- automatyczne wykrywanie aktywnych hostów w sieci lokalnej (discovery),
- skanowanie wybranych portów i usług sieciowych,
- identyfikację potencjalnych zagrożeń na podstawie reguł oraz wybranych skryptów NSE,
- cykliczne wykonywanie skanów przy użyciu harmonogramu (scheduler),
- zapisywanie wyników skanów na lokalnym nośniku danych (SATA lub USB),
- porównywanie wyników kolejnych skanów w celu wykrycia zmian (np. ndiff),
- generowanie raportów tekstowych podsumowujących stan sieci,
- wysyłkę raportów do administratora za pomocą poczty elektronicznej (SMTP).

System jest przeznaczony dla środowisk klasy SOHO (Small Office / Home Office) i został zaprojektowany z uwzględnieniem ograniczeń oraz specyfiki urządzeń MikroTik. Czyli przykładowo nie obejmuje on wykonywania działań destrukcyjnych, exploitacji podatności ani agresywnych testów penetracyjnych.

1.3 Motywacja i kontekst (operator telekomunikacyjny)

Współczesne sieci domowe oraz małe sieci firmowe coraz częściej zawierają wiele urządzeń podłączonych do Internetu, takich jak komputery, smartfony, urządzenia IoT czy systemy monitoringu. W wielu przypadkach użytkownicy końcowi nie posiadają wiedzy ani narzędzi umożliwiających ocenę poziomu bezpieczeństwa swojej sieci.

Operatorzy telekomunikacyjni, jako dostawcy infrastruktury sieciowej, posiadają potencjalną możliwość rozszerzenia swoich usług o funkcje związane z bezpieczeństwem. Jednym z podejść jest wykorzystanie routera jako punktu monitorowania i analizy sieci klienta.

Projekt wpisuje się w ten model, zakładając realizację systemu, który:

- działa bezpośrednio na routerze użytkownika,
- umożliwia zdalne zarządzanie oraz inicjowanie skanów,

- nie wymaga aktywnego udziału użytkownika końcowego,
- dostarcza czytelne informacje o stanie bezpieczeństwa sieci.

Zastosowanie kontenerów w systemie RouterOS pozwala na uruchamianie standardowych narzędzi linuxowych w izolowanym środowisku, co umożliwia implementację funkcji bezpieczeństwa bez konieczności ingerencji w podstawowe działanie routera. Dzięki temu możliwe jest stworzenie rozwiązania zbliżonego do usług operatorskich, przy jednoczesnym zachowaniu elastyczności i wykorzystaniu narzędzi open-source.

W kontekście projektu akademickiego rozwiązanie to pozwala na praktyczne poznanie współczesnych metod monitorowania bezpieczeństwa sieci, integracji systemów wbudowanych z narzędziami linuxowymi oraz projektowania systemów działających w warunkach ograniczonych zasobów sprzętowych.

2 Założenia systemu

2.0.1 Wymagania funkcjonalne

- **FR1:** Uruchomienie skanu z poziomu routera (harmonogram lub ręczny trigger).
- **FR2:** Wykonanie skanowania przy użyciu kontenera z Nmap (np. RouterOS v7 /container).
- **FR3:** Wykrycie hostów aktywnych w podsieci (discovery) oraz otwartych portów (port scan).
- **FR4:** Analiza podatności w modelu *best-effort*: NSE (np. kategorie safe) oraz reguły ryzyka (np. Telnet=wysokie).
- **FR5:** Wygenerowanie krótkiego raportu (preferowane: zmiany względem poprzedniego skanu).
- **FR6:** Lokalny zapis wyników i historii skanów na nośniku danych (SATA/USB).
- **FR7:** Wysyłka raportu do administratora za pomocą e-mail.

2.0.2 Wymagania niefunkcjonalne

- **NFR1 (zasoby sprzętowe):** System powinien działać na routerze MikroTik z RouterOS v7, wykorzystując kontenery oraz zewnętrzny nośnik danych (SATA lub USB). Zakłada się dostępność co najmniej 512 MB RAM oraz kilku GB przestrzeni dyskowej.
- **NFR2 (storage i trwałość danych):** Wyniki skanów oraz ich historia powinny być przechowywane lokalnie na routerze lub na zewnętrznym nośniku danych. System nie powinien wykorzystywać pamięci wbudowanej routera do długotrwałego przechowywania danych (128 MB pamięci konfiguracyjnej Mikrotika).
- **NFR3 (wydajność):** Proces skanowania nie powinien znacząco wpływać na działanie routera ani powodować zauważalnych opóźnień w obsłudze ruchu sieciowego. Przewidywane jest ograniczenie zakresu skanowania (np. wybrane porty) oraz czasu jego trwania.
- **NFR4 (automatyzacja):** System powinien działać automatycznie po instalacji, bez konieczności (ale z możliwością) ingerencji użytkownika końcowego. Wszystkie komponenty powinny być inicjalizowane poprzez minimum wykonawcze z poziomu SSH.
- **NFR5 (łatwość wdrożenia):** Instalacja systemu powinna być możliwa poprzez wykonanie jednej komendy, która pobiera i uruchamia skrypt konfiguracyjny RouterOS oraz inicjalizuje kontener.

- **NFR6 (modularność i rozszerzalność):** System powinien być zbudowany w sposób modułowy, umożliwiający łatwe rozszerzenie o dodatkowe funkcje, takie jak nowe profile skanowania, dodatkowe skrypty NSE lub inne mechanizmy analizy.
- **NFR7 (bezpieczeństwo):** System powinien wykorzystywać bezpieczne mechanizmy komunikacji dla raportowania (SMTP z TLS). Dane uwierzytelniające nie powinny być przechowywane w sposób jawny w kodzie.
- **NFR8 (nieinwazyjność):** System powinien wykorzystywać jedynie bezpieczne i nieinwazyjne metody skanowania (np. ograniczone skany portów, NSE typu safe). Nie powinien powodować zakłóceń w działaniu urządzeń w sieci lokalnej.
- **NFR9 (odporność na błędy):** System powinien obsługiwać sytuacje błędne, takie jak brak połączenia z Internetem lub błędy podczas skanowania. Błędy powinny być logowane, a próby wykonania operacji ponawiane.
- **NFR10 (utrzymanie i aktualizacja):** System powinien umożliwiać aktualizację kontenera oraz skryptów.

2.1 Ograniczenia platformy MikroTik oraz mechanizmu kontenerów

Projekt realizowany jest na platformie MikroTik z systemem RouterOS v7, z wykorzystaniem mechanizmu `/container`. Takie podejście pozwala uruchamiać narzędzia linuxowe bezpośrednio na urządzeniu brzegowym, jednak wiąże się również z szeregiem ograniczeń sprzętowych, systemowych i eksploatacyjnych.

2.1.1 Ograniczenia sprzętowe

Możliwość wykorzystania kontenerów w RouterOS zależy od architektury urządzenia oraz dostępnych zasobów. Pakiet `container` jest wspierany dla architektur `arm`, `arm64`, `x86` oraz `CHR`, a sama funkcja kontenerów wymaga zainstalowania dodatkowego pakietu systemowego. MikroTik wskazuje również, że użycie funkcji pobierania obrazu z rejestru (`remote-image`) wymaga znacznej ilości wolnej pamięci operacyjnej, a urządzenia z małą pamięcią flash mogą wymagać użycia wcześniej przygotowanych obrazów zapisanych na nośniku zewnętrznym. Dodatkowo producent zaleca stosowanie zewnętrznego nośnika danych, takiego jak SSD, HDD lub pamięć USB, ponieważ kontenery zużywają istotną ilość przestrzeni dyskowej, a szybkość nośnika wpływa na czas ekstrakcji obrazu i działania całego systemu.

Z tego względu w projekcie przyjęto, że środowisko testowe będzie wyposażone w zewnętrzny nośnik danych SATA lub USB oraz odpowiedni zapas pamięci RAM. Oznacza to, że rozwiązanie nie jest kierowane do najmniejszych urządzeń MikroTik z minimalną pamięcią masową, lecz do urządzeń umożliwiających praktyczne uruchamianie kontenerów w warunkach laboratoryjnych lub operatorskich.

2.1.2 Ograniczenia systemowe i wdrożeniowe

Mechanizm kontenerów w RouterOS jest domyślnie wyłączony i wymaga uprzedniego włączenia trybu obsługi kontenerów. MikroTik zaznacza, że do aktywacji tej funkcji wymagana jest fizyczna dostępność urządzenia. Po włączeniu tej funkcji kontenery mogą być już następnie konfigurowane i uruchamiane zdalnie. Oznacza to, że pełna transparentność pierwszego wdrożenia zależy od wcześniejszego przygotowania urządzenia co zostało dokładnie ustalone podczas konsultacji.

W praktyce oznacza to, że wymaganie instalacji rozwiązania pojedynczą komendą przez SSH należy rozumieć jako instalację logicznej części projektu na urządzeniu, na którym pakiet `container` został już wcześniej zainstalowany, a tryb obsługi kontenerów został uprzednio aktywowany. Przyjęcie takiego założenia jest uzasadnione w scenariuszu operatorskim, w którym urządzenie brzegowe jest wcześniej przygotowane do wdrożenia dodatkowych usług.

2.1.3 Ograniczenia bezpieczeństwa

MikroTik bardzo wyraźnie wskazuje, że uruchamianie kontenerów zwiększa powierzchnię ataku urządzenia. Producent zaznacza, że bezpieczeństwo routera zależy od bezpieczeństwa uruchamianego obrazu kontenera, a uruchamianie zewnętrznych obrazów może prowadzić do dodatkowych ryzyk, w tym możliwości eskalacji uprawnień lub pozostawienia trwałych zmian w systemie. Z tego powodu kontenery w RouterOS należy traktować jako funkcję wymagającą ostrożnego doboru obrazów, minimalizacji uprawnień oraz kontroli źródła pochodzenia oprogramowania.

W projekcie ograniczono to ryzyko poprzez zastosowanie własnego, możliwie lekkiego obrazu opartego o Alpine Linux oraz ograniczenie funkcji kontenera wyłącznie do realizacji procesu skanowania i generowania raportów. Dodatkowo zakłada się stosowanie wyłącznie nieinwazyjnych metod analizy sieci oraz bezpiecznych profili skanowania.

2.1.4 Ograniczenia wydajnościowe

Uruchomienie kontenera, ekstrakcja obrazu, działanie narzędzia Nmap oraz operacje zapisu wyników mogą wpływać na obciążenie procesora, pamięci RAM oraz nośnika danych routera. MikroTik udostępnia mechanizm `ram-high`, który pozwala ograniczyć zużycie pamięci przez kontenery, jednak należy traktować go jako środek pomocniczy, a nie pełne rozwiązanie problemu wydajności. Producent zaleca także stosowanie szybkich nośników danych, ponieważ wolniejsze urządzenia znacząco wydłużają czas ekstrakcji obrazu oraz działania kontenerów.

Z tego względu projekt nie zakłada pełnoskalowego, ciągłego skanowania wszystkich portów i wszystkich hostów w dużych sieciach. Zamiast tego przyjęto profil skanowania dostosowany do środowiska SOHO, obejmujący wybrane zakresy adresów, najistotniejsze porty oraz ostrożny dobór skryptów NSE. Takie podejście zmniejsza obciążenie urządzenia i zwiększa stabilność działania systemu.

2.1.5 Konsekwencje projektowe

Uwzględniając powyższe ograniczenia, w projekcie przyjęto następujące założenia:

- urządzenie testowe posiada zgodną architekturę i obsługę pakietu mikrotik `container`,
- pakiet `container` jest zainstalowany przed wdrożeniem właściwego rozwiązania,
- tryb obsługi kontenerów został wcześniej aktywowany,
- system korzysta z zewnętrznego nośnika danych o odpowiedniej pojemności i wydajności,
- obraz kontenera jest przygotowany wcześniej i zoptymalizowany pod kątem małego narzutu,
- zakres skanowania jest ograniczony do poziomu odpowiedniego dla urządzeń klasy SOHO.

Tak sformułowane założenia pozwalają zachować zgodność projektu z wymaganiami tematu, a jednocześnie uwzględniają rzeczywiste ograniczenia platformy MikroTik. Dzięki temu architektura rozwiązania pozostaje technicznie wiarygodna nawet w przypadku różnic pomiędzy środowiskiem projektowym a docelowym środowiskiem wdrożeniowym.

3 Ogólna architektura systemu

3.1 Opis koncepcji rozwiązania

Projektowany system opiera się na architekturze lokalnej, w której wszystkie komponenty odpowiedzialne za skanowanie, analizę oraz raportowanie działają bezpośrednio na routerze MikroTik. Router pełni rolę centralnego elementu systemu, zarządzającego procesem skanowania oraz przechowywaniem wyników.

Kluczowym elementem rozwiązania jest wykorzystanie mechanizmu kontenerów dostępnego w systemie RouterOS v7. W kontenerze uruchamiane jest lekkie środowisko np. Alpine Linux, zawierające narzędzie Nmap oraz skrypty odpowiedzialne za realizację procesu skanowania i analizy wyników.

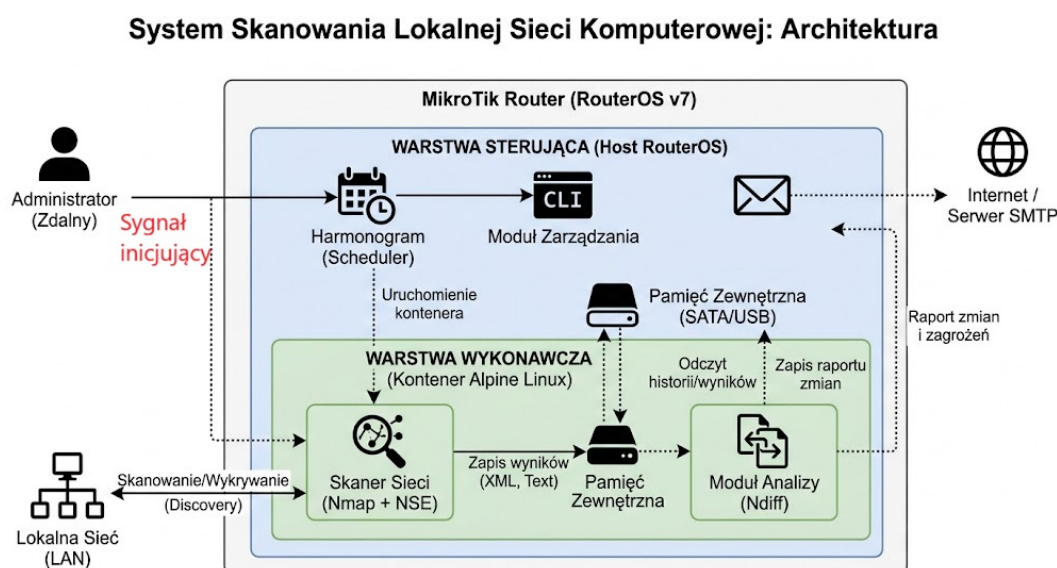
System składa się z dwóch głównych warstw:

- warstwy sterującej (RouterOS), odpowiedzialnej za zarządzanie, harmonogram oraz integrację z routerem,
- warstwy wykonawczej (kontener), odpowiedzialnej za skanowanie sieci oraz przetwarzanie wyników.

Router cyklicznie uruchamia kontener, który wykonuje skan sieci lokalnej, zapisuje wyniki na zewnętrznym nośniku danych oraz generuje raport. Następnie raport jest przekazywany do administratora za pomocą poczty elektronicznej.

System działa w pełni autonomicznie po instalacji, a jego wdrożenie odbywa się prostym poleceniem z poziomu SSH, co spełnia wymagania minimalnej ingerencji użytkownika końcowego.

3.2 Schemat architektury



Rysunek 1: Ogólna architektura systemu

3.3 Przepływ danych w systemie

Przepływ danych w systemie ma charakter lokalny i odbywa się w obrębie routera MikroTik oraz uruchomionego na nim kontenera.

Proces działania systemu można przedstawić w następujących krokach:

1. Harmonogram (scheduler) w systemie RouterOS uruchamia skrypt główny.
2. Skrypt inicjuje uruchomienie kontenera np. Alpine Linux.
3. Kontener wykonuje skan sieci lokalnej przy użyciu narzędzia Nmap.
4. Wyniki skanowania zapisywane są w formacie XML lub JSON oraz tekstowym na zewnętrznym nośniku danych.

5. System porównuje aktualny wynik z poprzednim skanem (np. przy użyciu narzędzia ndiff).
6. Generowany jest raport zawierający zmiany oraz potencjalne zagrożenia.
7. Router odczytuje raport i wysyła go do administratora za pomocą e-mail.

Dane nie są przesyłane do zewnętrznych systemów analitycznych, co upraszcza architekturę oraz zwiększa niezależność rozwiązania. Cała historia skanów przechowywana jest lokalnie na nośniku danych podłączonym do routera.

3.4 Model działania (lokalny vs zdalny)

System został zaprojektowany w modelu lokalnym, jednak z uwzględnieniem możliwości zdalnego zarządzania, co odpowiada podejściu stosowanemu przez operatorów telekomunikacyjnych.

3.4.1 Tryb lokalny

W trybie lokalnym wszystkie operacje wykonywane są bezpośrednio na routerze MikroTik. Obejmuje to:

- uruchamianie kontenera,
- skanowanie sieci,
- analizę wyników,
- przechowywanie danych,
- generowanie raportów.

Dzięki temu system nie wymaga dodatkowej infrastruktury i może działać niezależnie od dostępności usług zewnętrznych.

3.4.2 Tryb zdalny

System umożliwia również zdalne zarządzanie poprzez:

- dostęp do routera przez SSH,
- ręczne uruchomienie skanowania,
- zmianę konfiguracji systemu,
- aktualizację kontenera i skryptów.

W tym modelu operator lub administrator może inicjować działania bez udziału użytkownika końcowego, co odpowiada scenariuszowi nadzoru operatorskiego.

3.4.3 Porównanie podejść

- model lokalny zapewnia prostotę, niezależność i brak konieczności przesyłania danych,
- model zdalny umożliwia centralne zarządzanie i większą kontrolę nad systemem.

W projekcie przyjęto podejście hybrydowe, w którym przetwarzanie danych odbywa się lokalnie, natomiast sterowanie systemem może być realizowane zdalnie.

4 Komponenty systemu

4.1 Przegląd komponentów

System został zaprojektowany w sposób modułowy, z podziałem na logiczne komponenty odpowiedzialne za poszczególne etapy przetwarzania danych. Taki podział pozwala na łatwiejsze zarządzanie rozwiązaniem, jego rozwój oraz dostosowanie do ograniczeń platformy MikroTik.

Architektura systemu obejmuje następujące główne komponenty:

- **Moduł skanowania sieci** – odpowiedzialny za wykrywanie hostów oraz otwartych portów przy użyciu narzędzia Nmap oraz wybranych skryptów NSE.
- **Moduł analizy wyników** – realizuje podstawowe przetwarzanie danych ze skanów oraz identyfikację zmian pomiędzy kolejnymi pomiarami.
- **Moduł raportowania** – generuje uproszczone raporty oraz odpowiada za ich przekazanie do administratora (np. poprzez e-mail).
- **Moduł zarządzania** – umożliwia inicjalizację systemu, jego konfigurację oraz ręczne uruchamianie procesu skanowania.
- **Moduł harmonogramu** – odpowiada za cykliczne uruchamianie skanów zgodnie z przyjętym interwałem czasowym.
- **Warstwa kontenera** – zapewnia izolowane środowisko uruchomieniowe dla narzędzi skanujących i logiki przetwarzania.

Podział na komponenty ma charakter logiczny i nie musi w pełni odpowiadać fizycznemu rozmieszczeniu elementów systemu. W zależności od możliwości środowiska wykonawczego oraz przyjętej implementacji, niektóre funkcje mogą być realizowane wspólnie lub w sposób uproszczony.

Takie podejście pozwala zachować elastyczność projektu oraz dostosować go do rzeczywistych ograniczeń platformy sprzętowej i systemowej.

4.2 Moduł skanowania sieci (Nmap + NSE)

Moduł skanowania sieci odpowiada za identyfikację aktywnych hostów w sieci lokalnej oraz wykrywanie otwartych portów i usług. W projekcie wykorzystano narzędzie Nmap wraz z wybranymi skryptami NSE, które umożliwiają rozszerzenie funkcjonalności skanowania o podstawową analizę bezpieczeństwa.

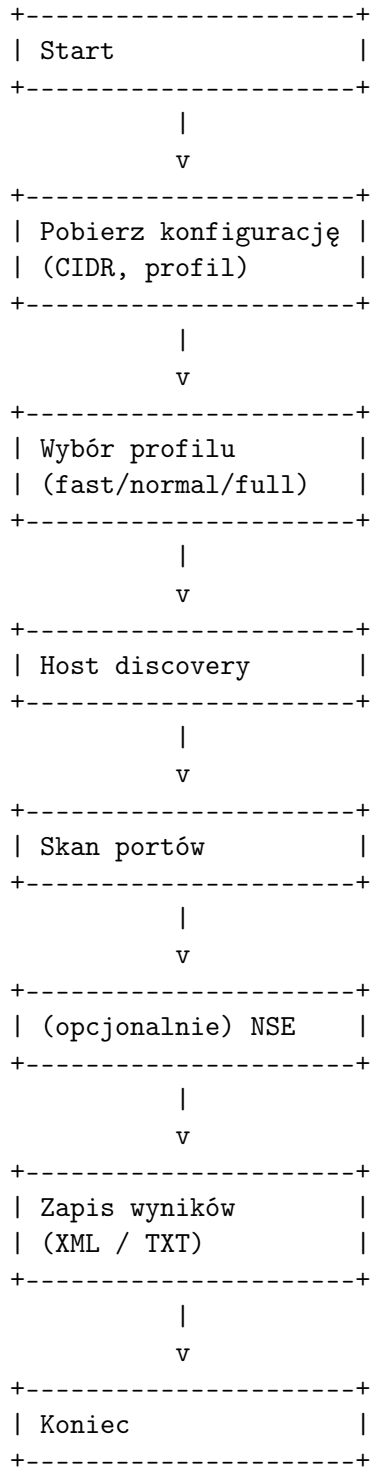
W celu dostosowania działania systemu do różnych scenariuszy oraz ograniczeń wydajnościowych routera, wprowadzono podział na profile skanowania o różnym poziomie szczegółowości:

- **Skan szybki (automatyczny)** – uruchamiany cyklicznie przez harmonogram. Obejmuje podstawowe wykrywanie hostów oraz ograniczony zakres portów. Jego celem jest szybkie wykrycie zmian w sieci przy minimalnym obciążeniu systemu.
- **Skan standardowy** – realizuje pełniejsze skanowanie wybranych portów oraz opcjonalną detekcję usług. Może być wykorzystywany jako tryb domyślny lub uruchamiany ręcznie.
- **Skan rozszerzony (dokładny)** – uruchamiany na żądanie użytkownika lub administratora. Obejmuje szerszy zakres portów, detekcję wersji usług oraz selektywne użycie skryptów NSE. Ze względu na większe obciążenie systemu stosowany będzie najprawdopodobniej sporadycznie.

Zakres skanowania został dostosowany do środowiska SOHO oraz ograniczeń sprzętowych routera. W szczególności uwzględniono:

- wykrywanie hostów (host discovery),
- skan wybranych portów,
- opcjonalną detekcję wersji usług,
- selektywne użycie skryptów NSE (kategorie safe).

4.2.1 Schemat działania modułu



4.2.2 Pseudokod algorytmu

```
INPUT:
  CIDR - zakres adresów
  PROFILE - fast / normal / full

BEGIN
  load configuration

  IF PROFILE == fast THEN
    set_ports(core_ports)
    nse_enabled = false
  ELSE IF PROFILE == normal THEN
    set_ports(extended_ports)
    enable_version_detection()
  ELSE IF PROFILE == full THEN
    set_ports(wide_range)
    enable_version_detection()
    enable_nse_scripts()
  ENDIF

  hosts = discover_hosts(CIDR)

  FOR each host IN hosts DO
    scan_ports(host)

    IF version_detection == true THEN
      detect_services(host)
    ENDIF

    IF nse_enabled == true THEN
      run_nse_scripts(host)
    ENDIF
  END FOR

  save_results()

END
```

4.2.3 Propozycja implementacji

Moduł skanowania może zostać zaimplementowany jako skrypt powłoki systemu Linux uruchamiany wewnątrz kontenera Alpine Linux. Skrypt przyjmuje parametr określający profil skanowania, co pozwala dynamicznie dostosować parametry narzędzia Nmap.

Przykładowe podejście implementacyjne obejmuje:

- przekazanie profilu jako argumentu (np. **fast**, **normal**, **full**),
- dobór parametrów Nmap w zależności od profilu,
- zapis wyników do plików (XML/TXT),
- możliwość rozszerzenia o dodatkowe profile lub parametry w przyszłości.

W zależności od możliwości urządzenia oraz przyjętej implementacji, zakres skanowania może być dodatkowo ograniczany lub rozszerzany. Takie podejście pozwala zachować elastyczność systemu oraz dostosować jego działanie do rzeczywistych warunków pracy.

4.3 Moduł analizy wyników

Moduł analizy wyników odpowiada za przetwarzanie danych uzyskanych w procesie skanowania oraz identyfikację zmian pomiędzy kolejnymi pomiarami. Jego głównym celem jest uproszczenie interpretacji wyników oraz wykrycie potencjalnych zagrożeń w sieci lokalnej.

Analiza obejmuje w szczególności:

- identyfikację nowych hostów w sieci,
- wykrycie zmian w dostępności usług (otwarte/zamknięte porty),
- identyfikację potencjalnie niebezpiecznych usług (np. Telnet, SMB),
- porównanie wyników bieżącego skanu z poprzednim stanem.

4.3.1 Schemat działania modułu



```

+-----+
| Generacja danych |
| do raportu      |
+-----+
      |
      v
+-----+
| Zapis aktualnego |
| stanu            |
+-----+
      |
      v
+-----+
| Koniec           |
+-----+

```

4.3.2 Pseudokod algorytmu

INPUT:

```

    current_scan
    previous_scan

```

BEGIN

```

    load current_scan

```

```

    IF previous_scan exists THEN
        load previous_scan

```

```

    ELSE

```

```

        previous_scan = empty

```

```

    ENDIF

```

```

    new_hosts = compare_hosts(current_scan, previous_scan)

```

```

    changed_ports = compare_ports(current_scan, previous_scan)

```

```

    risks = []

```

```

    FOR each service IN current_scan DO

```

```

        IF service IN risky_services THEN
            risks.add(service)

```

```

        ENDIF

```

```

    END FOR

```

```

    prepare_report_data(new_hosts, changed_ports, risks)

```

```

    save current_scan as previous_scan

```

END

4.3.3 Propozycja implementacji

Moduł analizy może zostać zaimplementowany na kilka sposobów, w zależności od przyjętego poziomu złożoności oraz dostępnych zasobów systemowych.

Najprostsze podejście obejmuje:

- wykorzystanie narzędzi systemowych (np. `diff`, `grep`, `awk`) do porównywania wyników,
- zapis wyników w formacie tekstowym i ich bezpośrednią analizę.

Bardziej zaawansowane podejście może obejmować:

- wykorzystanie języka skryptowego do parsowania wyników (np. XML z Nmap),
- budowę struktury danych reprezentującej stan sieci,
- bardziej rozbudowaną analizę zmian i klasyfikację ryzyka.

W zależności od możliwości środowiska oraz przyjętej implementacji, moduł analizy może być zrealizowany jako:

- osobny skrypt uruchamiany po zakończeniu skanowania,
- część skryptu skanującego,
- moduł napisany w języku skryptowym (np. Python) lub z wykorzystaniem narzędzi systemowych.

Zakres analizy może być dostosowywany do ograniczeń wydajnościowych urządzenia oraz wymagań projektu, co pozwala na zachowanie elastyczności rozwiązania.

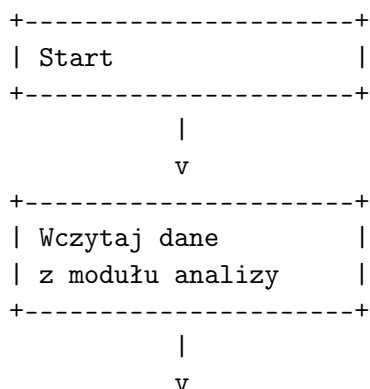
4.4 Moduł raportowania (email)

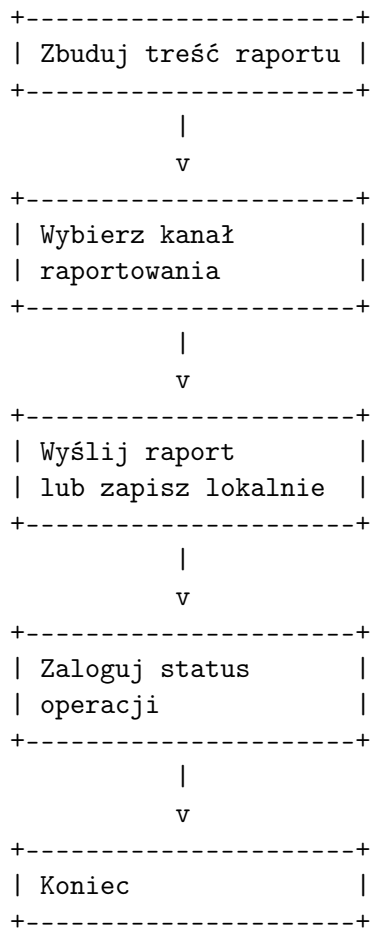
Moduł raportowania odpowiada za przygotowanie podsumowania wyników skanowania oraz przekazanie go administratorowi systemu. Jego zadaniem nie jest prezentowanie pełnych, surowych danych ze skanera, lecz wygenerowanie czytelnej i skróconej informacji o stanie sieci oraz o wykrytych zmianach lub potencjalnych zagrożeniach.

Zakres raportu może obejmować w szczególności:

- listę aktywnych hostów wykrytych w sieci,
- informacje o nowych hostach lub hostach niedostępnych względem poprzedniego skanu,
- wykryte otwarte porty i usługi,
- listę potencjalnie ryzykownych usług,
- ogólne podsumowanie wyniku skanowania.

4.4.1 Schemat działania modułu





4.4.2 Pseudokod algorytmu

```

INPUT:
    analysis_result
    report_channel

BEGIN
    load analysis_result

    report = build_summary(analysis_result)

    IF report_channel == email THEN
        send_email(report)
    ELSE
        save_report_locally(report)
    ENDIF

    log_status()

END

```

4.4.3 Propozycja implementacji

Moduł raportowania może zostać zrealizowany na kilka sposobów, w zależności od przyjętego modelu integracji pomiędzy RouterOS a kontenerem.

Najprostsze podejście zakłada wygenerowanie raportu tekstowego w kontenerze, zapisanie go na współdzielonym nośniku danych oraz wysłanie wiadomości e-mail z poziomu RouterOS z wykorzystaniem natywnego mechanizmu `/tool e-mail`. Takie rozwiązanie pozwala wykorzystać wbudowane funkcje systemu oraz ograniczyć złożoność implementacji.

Alternatywnie raport może być generowany i wysyłany bezpośrednio z poziomu kontenera, z wykorzystaniem narzędzi systemowych lub biblioteki języka skryptowego. W takim wariancie kontener przejmuje zarówno budowę raportu, jak i jego dostarczenie do odbiorcy.

W zależności od przyjętej implementacji raport może mieć postać:

- prostego tekstu,
- wiadomości e-mail zawierającej skrócone podsumowanie,
- pliku archiwizowanego lokalnie i wykorzystywanego do późniejszej analizy.

Ostateczna forma raportowania może zostać dostosowana do ograniczeń środowiska wykonawczego, wymagań funkcjonalnych oraz dostępnych mechanizmów komunikacyjnych.

4.5 Moduł zarządzania (CLI/SSH)

Moduł zarządzania odpowiada za konfigurację systemu, jego inicjalizację oraz ręczne sterowanie procesem skanowania. Umożliwia również dostosowanie parametrów pracy systemu bez konieczności modyfikacji kodu źródłowego.

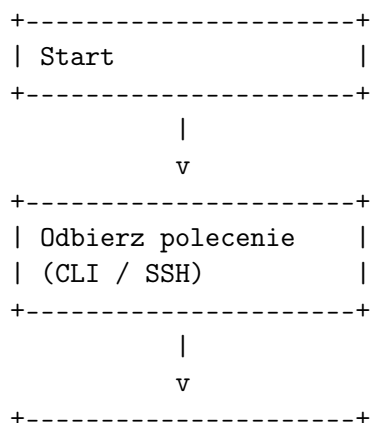
W kontekście projektu głównym mechanizmem zarządzania jest interfejs wiersza poleceń (CLI) dostępny poprzez system RouterOS oraz dostęp SSH. Pozwala to na realizację scenariusza operatorskiego, w którym administrator może zdalnie zarządzać urządzeniem brzegowym bez udziału użytkownika końcowego.

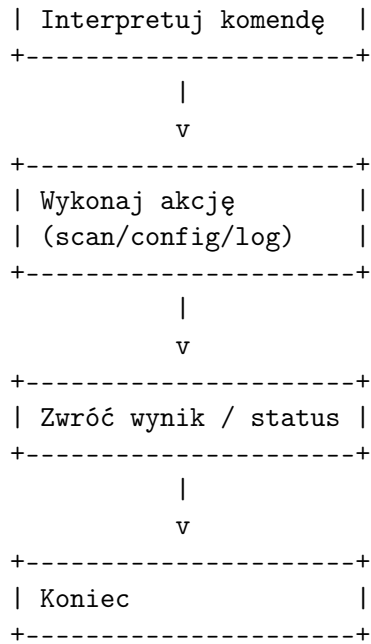
4.5.1 Zakres funkcjonalny

Moduł zarządzania umożliwia:

- inicjalizację systemu (instalacja i konfiguracja),
- ręczne uruchamianie skanów (wybór profilu),
- zmianę parametrów skanowania (np. zakres adresów, porty),
- zarządzanie harmonogramem,
- podgląd logów oraz statusu systemu.

4.5.2 Schemat działania modułu





4.5.3 Pseudokod algorytmu

INPUT:

command

BEGIN

read command

IF command == "scan fast" THEN

run_scan(profile="fast")

ELSE IF command == "scan normal" THEN

run_scan(profile="normal")

ELSE IF command == "scan full" THEN

run_scan(profile="full")

ELSE IF command == "status" THEN

show_status()

ELSE IF command == "config" THEN

update_configuration()

ENDIF

return result

END

4.5.4 Propozycja implementacji

Moduł zarządzania może zostać zaimplementowany jako zestaw skryptów RouterOS, które realizują określone funkcje systemu. Skrypty te mogą być wywoływane bezpośrednio z poziomu CLI lub poprzez zdalne połączenie SSH.

Przykładowe podejście obejmuje:

- skrypt instalacyjny (`install.rsc`),
- skrypt główny uruchamiający skan,

- skrypty pomocnicze do konfiguracji i zarządzania,
- wykorzystanie mechanizmu `/system script` oraz `/system scheduler`.

W zależności od przyjętej implementacji, możliwe jest również rozszerzenie modułu o prosty interfejs API (np. HTTP), jednak nie jest to wymagane do poprawnego działania systemu i traktowane jest jako opcjonalne rozszerzenie.

Takie podejście pozwala na zachowanie zgodności z wymaganiami projektu dotyczącymi zdalnego zarządzania oraz minimalnej ingerencji użytkownika końcowego.

4.6 Moduł harmonogramu (scheduler)

Moduł harmonogramu odpowiada za automatyczne uruchamianie procesu skanowania w określonych odstępach czasu. Jego głównym zadaniem jest zapewnienie cyklicznego działania systemu bez konieczności ingerencji użytkownika końcowego.

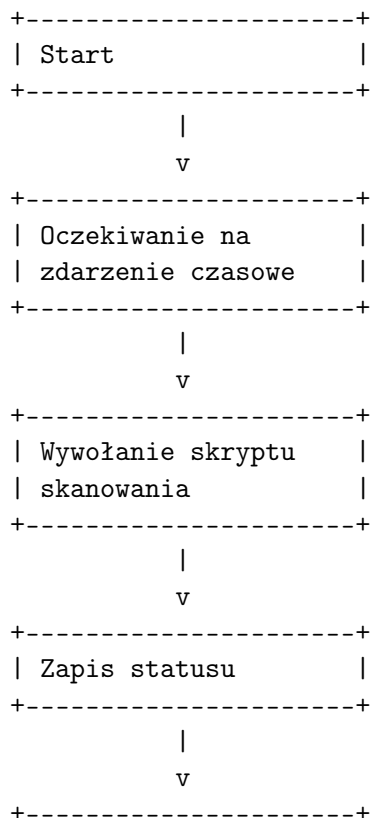
W projekcie planowane jest wykorzystanie natywnego mechanizmu `/system scheduler` dostępnego w RouterOS, który umożliwia wykonywanie zadań według ustalonego harmonogramu.

4.6.1 Zakres funkcjonalny

Moduł harmonogramu umożliwia:

- cykliczne uruchamianie skanów (np. co określony czas),
- wybór profilu skanowania dla zadań automatycznych (np. skan szybki),
- współpracę z modułem zarządzania w zakresie ręcznego uruchamiania skanów,
- możliwość modyfikacji interwału działania.

4.6.2 Schemat działania modułu



```
| Powrót do oczekiwania|  
+-----+
```

4.6.3 Pseudokod algorytmu

INPUT:

```
    interval  
    profile = fast
```

BEGIN

```
    WAIT interval  
  
    run_scan(profile)  
  
    log_execution()
```

REPEAT

END

4.6.4 Propozycja implementacji

Moduł harmonogramu może zostać zaimplementowany z wykorzystaniem mechanizmu `/system scheduler` w RouterOS. Scheduler uruchamia wskazany skrypt w określonych odstępach czasu, co pozwala na automatyczne wywoływanie procesu skanowania.

Przykładowe podejście obejmuje:

- utworzenie wpisu w schedulerze z określonym interwałem,
- wywołanie skryptu uruchamiającego skan,
- ustawienie profilu skanowania (np. szybki) dla zadań automatycznych.

W zależności od przyjętej implementacji możliwe jest wykorzystanie jednego lub wielu wpisów harmonogramu (np. dla różnych profili skanowania). Takie podejście pozwala dostosować działanie systemu do wymagań oraz ograniczeń środowiska.

4.7 Kontener (Docker / LXC)

Kontener stanowi izolowane środowisko uruchomieniowe dla głównych narzędzi systemu, w szczególności narzędzia Nmap oraz skryptów odpowiedzialnych za skanowanie i podstawowe przetwarzanie wyników. Zastosowanie konteneryzacji pozwala oddzielić logikę aplikacyjną od systemu RouterOS oraz uprościć proces wdrożenia i aktualizacji rozwiązania.

W projekcie przyjęto wykorzystanie lekkiego obrazu opartego np. o Alpine Linux, zawierającego niezbędne narzędzia oraz skrypty uruchomieniowe. Kontener uruchamiany jest z poziomu RouterOS z wykorzystaniem mechanizmu `/container`.

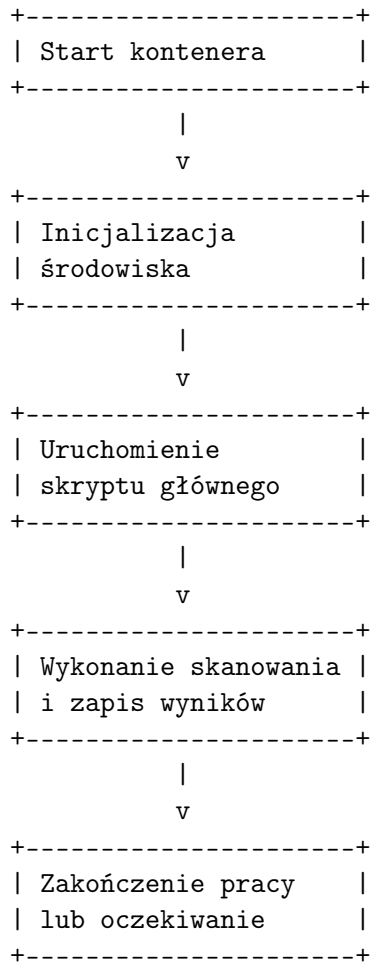
4.7.1 Zakres funkcjonalny

Warstwa kontenera odpowiada za:

- uruchomienie środowiska skanującego,
- wykonanie procesu skanowania,
- zapis wyników do plików,

- przekazanie danych do dalszej analizy lub raportowania,
- separację narzędzi linuxowych od systemu RouterOS.

4.7.2 Schemat działania modułu



4.7.3 Pseudokod algorytmu

```

INPUT:
    scan_profile
    target_network

BEGIN
    initialize container environment

    load configuration(scan_profile, target_network)

    run main scanning script

    save results

    exit or wait for next execution

END

```

4.7.4 Propozycja implementacji

Kontener może zostać przygotowany jako obraz (Docker) oparty o np. Alpine Linux, zawierający narzędzie Nmap, wybrane skrypty NSE oraz skrypty pomocnicze odpowiedzialne za uruchomienie procesu skanowania, zapis wyników i inne potrzebne funkcje. Obraz może być budowany lokalnie lub udostępniany z poziomu repozytorium projektu.

W zależności od przyjętego wariantu wdrożenia możliwe jest:

- uruchamianie pojedynczego kontenera wykonującego cały proces,
- podział logiki pomiędzy skrypty wywoływane przez RouterOS i skrypty działające wewnątrz kontenera,
- wykorzystanie kontenera jako środowiska jednorazowego uruchomienia lub komponentu działającego cyklicznie.

W projekcie kontener traktowany jest jako główna warstwa wykonawcza dla narzędzi linuxowych. Ostateczny sposób organizacji jego pracy może zostać dostosowany do ograniczeń środowiska testowego oraz do przyjętego modelu integracji z RouterOS.

5 Interfejsy i komunikacja

5.1 Komunikacja między modułami

System składa się z dwóch głównych warstw: warstwy RouterOS oraz warstwy kontenera. Komunikacja między nimi odbywa się lokalnie, bez udziału zewnętrznych usług.

RouterOS odpowiada za sterowanie procesem, natomiast kontener realizuje właściwe operacje skanowania i analizy. Komunikacja przebiega w następujący sposób:

- RouterOS uruchamia kontener za pomocą mechanizmu `/container`,
- kontener wykonuje skan sieci oraz zapisuje wyniki do systemu plików,
- wyniki są zapisywane na Routerze/ współdzielonym nośniku danych (SATA/USB), lub w kontenerze,
- RouterOS odczytuje wygenerowane raporty z systemu plików,
- RouterOS realizuje dalsze działania (np. wysyłka e-mail).

Wymiana danych odbywa się poprzez pliki (np. XML, TXT), co upraszcza implementację oraz eliminuje potrzebę stosowania dodatkowych interfejsów komunikacyjnych.

5.2 Interfejs wejściowy (konfiguracja)

System posiada prosty interfejs konfiguracyjny realizowany z poziomu RouterOS. Konfiguracja odbywa się poprzez skrypty oraz zmienne definiowane podczas instalacji systemu.

Do podstawowych parametrów konfiguracyjnych należą:

- zakres adresów IP do skanowania (np. 192.168.1.0/24),
- profil skanowania (np. lista portów, tryb skanowania),
- harmonogram wykonywania skanów,
- dane serwera SMTP (adres, port, użytkownik),
- adres e-mail odbiorcy raportów.

Konfiguracja może być realizowana poprzez:

- jednorazowy skrypt instalacyjny (`.rsc`),
- ręczną edycję parametrów w RouterOS,
- zdalny dostęp przez SSH.

Takie podejście pozwala na łatwe dostosowanie systemu bez konieczności ingerencji w kod kontenera.

5.3 Interfejs wyjściowy (raporty)

System generuje raporty zawierające wyniki skanowania oraz wykryte zmiany w sieci. Raporty mają formę tekstową, co zapewnia ich czytelność oraz kompatybilność z różnymi systemami.

Raport zawiera np. m.in.:

- listę aktywnych hostów,
- wykryte otwarte porty i usługi,
- nowe elementy w porównaniu do poprzedniego skanu,
- potencjalne zagrożenia (np. otwarte porty wysokiego ryzyka),
- podsumowanie stanu sieci.

Raporty są:

- zapisywane lokalnie na nośniku danych,
- wysyłane do administratora za pomocą e-mail (SMTP).

Dzięki temu administrator ma zarówno dostęp do historii skanów, jak i bieżące informacje o zmianach w sieci.

5.4 Integracja z routerem MikroTik

System jest ściśle zintegrowany z routerem MikroTik oraz systemem RouterOS v7. Integracja obejmuje wykorzystanie natywnych mechanizmów systemowych:

- `/container` – uruchamianie i zarządzanie kontenerem,
- `/system scheduler` – cykliczne uruchamianie skanów,
- `/system script` – logika sterująca systemem,
- `/tool e-mail` – wysyłka raportów,
- system plików RouterOS – przechowywanie wyników.

Router pełni rolę centralnego punktu integracji, który:

- inicjuje działanie systemu,
- zarządza harmonogramem,
- kontroluje kontener,
- obsługuje komunikację z użytkownikiem (raporty).

Takie podejście pozwala na pełne wykorzystanie możliwości RouterOS oraz minimalizuje potrzebę stosowania dodatkowych komponentów zewnętrznych.

5.4.1 Algorytm działania

Integracja systemu z routerem MikroTik realizowana jest poprzez zestaw skryptów RouterOS oraz mechanizm kontenerów. Proces działania rozpoczyna się od wywołania skryptu głównego przez harmonogram systemowy (`/system scheduler`) lub ręcznie przez administratora.

Skrypt sprawdza dostępność kluczowych komponentów, takich jak pakiet `/container`, dostępność nośnika danych oraz konfiguracja środowiska. Następnie inicjowane jest uruchomienie kontenera odpowiedzialnego za skanowanie sieci.

RouterOS nadzoruje działanie kontenera oraz oczekuje na zakończenie procesu skanowania. Po jego zakończeniu system odczytuje wygenerowane pliki raportów z systemu plików.

W kolejnym kroku wykonywana jest operacja wysyłki raportu przy użyciu mechanizmu `/tool e-mail`. W przypadku błędów system zapisuje odpowiednie wpisy w logach RouterOS.

Cały proces jest cyklicznie powtarzany zgodnie z konfiguracją harmonogramu.

5.4.2 Pseudokod

```
START

# Wywołanie przez scheduler lub administratora

Sprawdź czy pakiet /container jest dostępny
Sprawdź czy kontener istnieje

IF kontener nie istnieje THEN
    Zaloguj błąd
    STOP
END IF

Sprawdź dostęp do nośnika danych

IF brak dostępu do storage THEN
    Zaloguj błąd
    STOP
END IF

Uruchom kontener ( /container start )

Czekaj na zakończenie pracy kontenera

IF brak pliku raportu THEN
    Zaloguj błąd
    STOP
END IF

Odczytaj raport z systemu plików

IF konfiguracja e-mail poprawna THEN
    Wyslij raport (/tool e-mail)
ELSE
    Zaloguj brak konfiguracji SMTP
END IF

Zapisz informacje o wykonaniu zadania

END
```

6 Mechanizm skanowania sieci

6.1 Zakres skanowania

Zakres skanowania został dobrany z uwzględnieniem ograniczeń sprzętowych routera MikroTik oraz charakterystyki sieci klasy SOHO. System nie wykonuje pełnego skanowania wszystkich portów, lecz koncentruje się na wybranych usługach o największym znaczeniu z punktu widzenia bezpieczeństwa.

Skanowanie obejmuje:

- wykrywanie aktywnych hostów w podsieci (discovery),
- analizę wybranych portów TCP,
- identyfikację usług działających na wykrytych portach.

Domyślny zestaw portów obejmuje:

- 21 (FTP),
- 22 (SSH),
- 23 (Telnet),
- 53 (DNS),
- 80 (HTTP),
- 443 (HTTPS),
- 445 (SMB),
- 3389 (RDP),
- 8291 (Winbox - MikroTik).

Takie podejście pozwala ograniczyć czas skanowania oraz obciążenie systemu, przy jednoczesnym zachowaniu wysokiej skuteczności wykrywania potencjalnych zagrożeń.

6.2 Wykorzystanie Nmap

Do realizacji procesu skanowania wykorzystano narzędzie Nmap uruchamiane wewnątrz kontenera np. Alpine Linux. Nmap umożliwia efektywne wykrywanie hostów, skanowanie portów oraz identyfikację usług sieciowych.

W projekcie wykorzystano następujące funkcje Nmap:

- skanowanie portów TCP (-p),
- detekcja wersji usług (-sV),
- wykrywanie hostów aktywnych (-sn),
- zapis wyników w formacie XML (-oX),
- zapis wyników w formacie tekstowym.

Przykładowe polecenie dla systemu:

```
nmap -sV -p 21,22,23,53,80,443,445,3389,8291 \
-oX scan.xml 192.168.1.0/24
```

Zastosowanie Nmap pozwala na uzyskanie szczegółowych informacji o stanie sieci przy relatywnie niskim narzucie obliczeniowym.

6.3 Wykorzystanie NSE (Nmap Scripting Engine)

Nmap Scripting Engine (NSE) umożliwia rozszerzenie funkcjonalności skanowania poprzez wykorzystanie gotowych skryptów analizujących konkretne usługi i potencjalne podatności.

W projekcie zastosowano wyłącznie skrypty należące do kategorii bezpiecznych (ang. *safe*), które nie ingerują w działanie systemów i nie powodują zakłóceń w sieci.

Przykładowe zastosowania NSE:

- identyfikacja podatnych usług,
- wykrywanie słabych konfiguracji,
- analiza dostępności usług.

Przykładowe użycie NSE:

```
nmap -sV --script safe -p 80,443 192.168.1.0/24
```

Ograniczenie się do bezpiecznych skryptów NSE pozwala zachować zgodność z założeniem nieinwazyjności systemu.

6.4 Optymalizacja skanowania

Ze względu na ograniczone zasoby routera MikroTik konieczne będzie zastosowanie mechanizmów optymalizujących proces skanowania.

W projekcie planowano zastosowanie następującego podejścia:

- ograniczenie liczby skanowanych portów do najważniejszych,
- stosowanie skanowania tylko wybranych podsieci,
- unikanie agresywnych trybów skanowania (np. **-A**),
- stosowanie bezpiecznych skryptów NSE,
- wykonywanie skanów w określonych odstępach czasu (scheduler),
- zapisywanie wyników lokalnie zamiast przesyłania dużych ilości danych.

Dodatkowo możliwe jest zastosowanie parametrów ograniczających intensywność skanowania, takich jak:

- **-T2** lub **-T3** – kontrola prędkości skanowania,
- **-max-retries** – ograniczenie liczby powtórzeń,
- **-host-timeout** – ograniczenie czasu skanowania pojedynczego hosta.

Takie podejście pozwala zminimalizować wpływ systemu na działanie sieci oraz zapewnić stabilność pracy routera.

7 Mechanizm detekcji zagrożeń

7.1 Typy wykrywanych zagrożeń

System wykrywa potencjalne zagrożenia na podstawie wyników skanowania sieci oraz analizy dostępnych usług i portów. Wykrywanie ma charakter heurystyczny i opiera się na zestawie zdefiniowanych reguł.

Do głównych typów wykrywanych zagrożeń należą:

- **otwarte porty wysokiego ryzyka** (np. Telnet, FTP),
- **nieznane urządzenia w sieci** (nowe hosty),
- **zmiany konfiguracji usług** (np. pojawienie się nowych portów),
- **niezabezpieczone usługi** (np. HTTP bez HTTPS),
- **usługi administracyjne dostępne w sieci lokalnej** (np. SSH, RDP),
- **potencjalnie podatne usługi wykryte przez NSE.**

System nie wykonuje aktywnej eksploatacji podatności, lecz identyfikuje symptomy mogące wskazywać na zwiększone ryzyko bezpieczeństwa.

7.2 Analiza wyników skanowania

Analiza wyników skanowania odbywa się lokalnie w kontenerze i polega na przetwarzaniu danych wygenerowanych przez Nmap w formacie XML lub tekstowym.

Proces analizy obejmuje:

- ekstrakcję listy hostów oraz ich adresów IP,
- identyfikację otwartych portów i usług,
- porównanie wyników z poprzednim skanem (np. przy użyciu `ndiff`),
- wykrywanie nowych hostów oraz nowych portów,
- identyfikację zanikających usług (potencjalne zmiany w sieci).

Wyniki analizy są następnie przekazywane do modułu detekcji zagrożeń, który interpretuje je w kontekście bezpieczeństwa.

7.3 Reguły detekcji

System wykorzystuje zestaw prostych reguł decyzyjnych przypisujących poziom ryzyka do wykrytych zdarzeń. Reguły te są łatwe do rozszerzenia i mogą być modyfikowane w zależności od potrzeb.

Przykładowe reguły detekcji:

- **Telnet (port 23) otwarty** → wysoki poziom ryzyka,
- **FTP (port 21) otwarty** → średni poziom ryzyka,
- **HTTP bez HTTPS** → średni poziom ryzyka,
- **nowy host w sieci** → potencjalne zagrożenie,

- **nowy otwarty port** → zmiana wymagająca uwagi,
- **RDP (port 3389)** → podwyższony poziom ryzyka,
- **wynik NSE wskazujący podatność** → wysoki poziom ryzyka.

Na podstawie spełnionych reguł system generuje końcowy raport zawierający:

- listę wykrytych zagrożeń,
- przypisany poziom ryzyka (niski / średni / wysoki),
- krótkie uzasadnienie dla każdej detekcji.

Takie podejście pozwala na uzyskanie czytelnych i użytecznych informacji dla administratora, bez konieczności stosowania złożonych systemów analitycznych.

8 Mechanizm raportowania

8.1 Format raportu

System generuje raporty tekstowe zawierające podsumowanie wyników skanowania oraz wykrytych zmian w sieci. Format raportu został zaprojektowany w sposób czytelny i zrozumiały dla administratora, bez konieczności analizy surowych danych Nmap.

Raport składa się z następujących sekcji:

- **Informacje ogólne:**
 - data i godzina skanowania,
 - zakres skanowanej sieci,
 - czas trwania skanowania.
- **Wykryte hosty:**
 - lista aktywnych adresów IP,
 - liczba hostów w sieci.
- **Otwarte porty i usługi:**
 - lista portów dla każdego hosta,
 - wykryte usługi (np. SSH, HTTP).
- **Zmiany względem poprzedniego skanu:**
 - nowe hosty,
 - nowe otwarte porty,
 - zniknięte usługi.
- **Wykryte zagrożenia:**
 - lista zdarzeń bezpieczeństwa,
 - poziom ryzyka (niski / średni / wysoki),
 - krótkie uzasadnienie.

Przykładowy fragment raportu:

```
Scan report - 2025-01-10 12:00

Network: 192.168.1.0/24
Hosts detected: 5

[!] New host detected: 192.168.1.25
[!] Port 23 (Telnet) open on 192.168.1.10 - HIGH RISK

Open services:
192.168.1.10 -> 22 (SSH), 23 (Telnet)
192.168.1.20 -> 80 (HTTP), 443 (HTTPS)
```

Raport zapisywany jest w formie pliku tekstowego na nośniku danych oraz wykorzystywany do wysyłki e-mail.

8.2 Wysyłka email

Wysyłka raportów realizowana jest za pomocą wbudowanego narzędzia RouterOS `/tool e-mail`. Mechanizm ten pozwala na automatyczne przekazywanie wyników skanowania do administratora.

Proces wysyłki obejmuje:

- konfigurację serwera SMTP (adres, port, użytkownik, hasło),
- przygotowanie treści wiadomości (raport),
- wysłanie e-mail z poziomu RouterOS.

Przykładowa konfiguracja:

```
/tool e-mail set address=smtp.example.com port=587 \
user="user@example.com" password="password" tls=starttls
```

Przykładowe wysłanie wiadomości:

```
/tool e-mail send to="admin@example.com" \
subject="Network Scan Report" \
body="Scan completed - see report"
```

W praktycznej implementacji treść raportu może być bezpośrednio wstawiana do pola `body` lub zapisywana jako plik i dołączana do wiadomości.

8.3 Automatyzacja powiadomień

System raportowania działa automatycznie dzięki wykorzystaniu mechanizmu harmonogramu RouterOS (`/system scheduler`).

Automatyzacja obejmuje:

- cykliczne uruchamianie skanowania (np. co godzinę),
- generowanie raportów bez udziału użytkownika,
- automatyczną wysyłkę e-mail po zakończeniu skanowania.

Dodatkowo możliwe jest wprowadzenie mechanizmu warunkowego wysyłania raportów, np. tylko w przypadku wykrycia zmian lub zagrożeń.

Przykładowe zastosowania:

- wysyłka raportu tylko przy nowych hostach,

- wysyłka alertu przy wykryciu wysokiego ryzyka,
- brak wysyłki przy braku zmian.

Takie podejście pozwala ograniczyć liczbę niepotrzebnych powiadomień oraz skupić uwagę administratora na istotnych zdarzeniach.

Cały proces raportowania działa w sposób autonomiczny i nie wymaga ingerencji użytkownika końcowego po zakończeniu instalacji systemu.

9 Wdrożenie systemu

9.1 Konteneryzacja (Docker / LXC)

System wykorzystuje mechanizm kontenerów dostępny w RouterOS v7, który umożliwia uruchamianie lekkich środowisk linuxowych bezpośrednio na routerze MikroTik. W projekcie planowano zastosowanie kontenera opartego o np. Alpine Linux, zawierającego narzędzie Nmap oraz skrypty realizujące proces skanowania i analizy.

Obraz kontenera przygotowywany jest przy użyciu standardowego pliku `Dockerfile`, a następnie importowany do RouterOS.

Przykładowa zawartość `Dockerfile`:

```
FROM alpine:latest

RUN apk add --no-cache nmap nmap-scripts bash

WORKDIR /app

COPY scan.sh /app/scan.sh
COPY analyze.sh /app/analyze.sh

CMD ["sh", "/app/scan.sh"]
```

Takie podejście pozwala na izolację środowiska wykonawczego oraz łatwą aktualizację komponentów systemu.

9.2 Struktura projektu (GitHub)

Projekt przechowywany jest w repozytorium GitHub i posiada modułową strukturę, umożliwiającą łatwe rozwijanie systemu.

Przykładowa struktura katalogów:

```
project/
  routeros/
    install.rsc
    main.rsc
    scheduler.rsc
  container/
    Dockerfile
    scan.sh
    analyze.sh
    report.sh
  data/
    scans/
    reports/
  README.md
```

Opis elementów:

- `routeros/` – skrypty konfigurujące router,
- `container/` – środowisko skanowania,
- `data/` – dane generowane przez system,
- `README.md` – instrukcja wdrożenia.

9.3 Instalacja przez SSH (one-command)

System został zaprojektowany w taki sposób, aby jego instalacja była możliwa przy użyciu pojedynczej komendy wykonanej z poziomu SSH.

Proces instalacji obejmuje:

- pobranie skryptu instalacyjnego,
- konfigurację środowiska RouterOS,
- utworzenie katalogów roboczych,
- dodanie kontenera,
- konfigurację harmonogramu.

Przykładowa komenda instalacyjna:

```
/tool fetch url="https://example.com/install.rsc" \
mode=https dst-path=install.rsc
/import file-name=install.rsc
```

Skrypt instalacyjny odpowiada za pełną konfigurację systemu, co minimalizuje ingerencję użytkownika końcowego.

9.4 Uruchomienie systemu

Po zakończeniu instalacji system działa automatycznie zgodnie z konfiguracją harmonogramu.

Uruchomienie może nastąpić w dwóch trybach:

- **automatyczne** – poprzez wpis w `/system scheduler`,
- **manualne** – poprzez ręczne uruchomienie skryptu.

Przykładowe ręczne uruchomienie:

```
/system script run main
```

Przykładowa konfiguracja harmonogramu:

```
/system scheduler add name=scan interval=1h \
on-event=main
```

System po uruchomieniu:

- inicjuje kontener,
- wykonuje skan sieci,
- analizuje wyniki,
- generuje raport,
- wysyła powiadomienie e-mail.

Dzięki temu rozwiązanie działa w pełni autonomicznie i nie wymaga dalszej ingerencji użytkownika po instalacji. Rozwiązanie może zostać łatwo wdrożone na wielu urządzeniach poprzez powielenie procesu instalacji, co odpowiada modelowi wdrożeń operatorskich.

10 Podsumowanie

10.1 Założone cele

Celem projektu jest zaprojektowanie systemu umożliwiającego automatyczne skanowanie lokalnej sieci komputerowej oraz wykrywanie potencjalnych zagrożeń bezpieczeństwa z wykorzystaniem narzędzi open-source.

W ramach projektu zakłada się:

- wykorzystanie narzędzia Nmap do realizacji skanowania sieci,
- zastosowanie konteneryzacji do izolacji środowiska wykonawczego,
- integrację rozwiązania z routerem MikroTik (RouterOS v7),
- opracowanie mechanizmu analizy wyników i detekcji zagrożeń,
- przygotowanie systemu raportowania oraz powiadomień e-mail,
- umożliwienie instalacji systemu przy użyciu pojedynczej komendy (one-command).

Projekt ma na celu odwzorowanie modelu działania operatora telekomunikacyjnego, w którym urządzenie brzegowe pełni rolę punktu monitorowania bezpieczeństwa sieci użytkownika końcowego.

10.2 Możliwości rozwoju

Projekt stanowi bazę do dalszego rozwoju i może zostać rozszerzony o dodatkowe funkcjonalności.

Potencjalne kierunki rozwoju obejmują:

- rozszerzenie zestawu reguł detekcji zagrożeń,
- wprowadzenie bardziej zaawansowanego systemu oceny ryzyka,
- generowanie raportów w formacie HTML lub PDF,
- integrację z zewnętrznymi systemami monitorowania,
- dodanie interfejsu webowego do zarządzania systemem,
- centralne zarządzanie wieloma urządzeniami (model operatorski).

10.3 Ograniczenia rozwiązania

Projekt uwzględnia ograniczenia wynikające z wykorzystania routera MikroTik jako platformy wykonawczej.

Do głównych ograniczeń należą:

- ograniczone zasoby sprzętowe urządzenia (CPU, RAM, Miejsce na dysku),
- ograniczona wydajność mechanizmu kontenerów,
- brak wsparcia dla zaawansowanych technik skanowania wymagających surowych pakietów,
- ograniczona możliwość analizy dużych ilości danych,
- zależność od poprawnej konfiguracji RouterOS oraz środowiska kontenerowego.

Pomimo tych ograniczeń projekt zakłada stworzenie rozwiązania funkcjonalnego i możliwego do wdrożenia w środowiskach klasy SOHO.

Literatura

- [1] Docker. Docker documentation. <https://docs.docker.com/>, 2024.
- [2] e-SUS / esus-it.pl. Router mikrotik rb5009upr+s+in – karta produktu. https://www.esus-it.pl/product-pol-92565-Router-Mikrotik-RB5009UPr-S-IN-7x-RJ-45-10-100-1000-Mb-s-1x.html?query_id=2. Dostęp: 2026-04-10.
- [3] Free Software Foundation. Gnu awk user’s guide. <https://www.gnu.org/software/gawk/manual/>, 2024.
- [4] Free Software Foundation. Gnu diffutils manual. <https://www.gnu.org/software/diffutils/manual/>, 2024.
- [5] Python Software Foundation. difflib — helpers for computing deltas (ndiff). <https://docs.python.org/3/library/difflib.html>, 2024.
- [6] IEEE. Posix awk specification. <https://pubs.opengroup.org/onlinepubs/9699919799/utilities/awk.html>, 2018.
- [7] Alpine Linux. Official documentation. <https://docs.alpinelinux.org/>, 2024.
- [8] MikroTik. Chr downloads. <https://mikrotik.com/download/chr>. Dostęp: 2026-04-06.
- [9] MikroTik. Container. <https://help.mikrotik.com/docs/spaces/ROS/pages/84901929/Container>, 2024.
- [10] MikroTik. E-mail tool. <https://help.mikrotik.com/docs/spaces/ROS/pages/24805377/E-mail>, 2024.
- [11] MikroTik. Packages. <https://help.mikrotik.com/docs/spaces/ROS/pages/40992872/Packages>, 2024.
- [12] MikroTik. Scheduler. <https://help.mikrotik.com/docs/spaces/ROS/pages/40992881/Scheduler>, 2024.
- [13] MikroTik. Ssh. <https://help.mikrotik.com/docs/spaces/ROS/pages/132350014/SSH>, 2024.
- [14] MikroTik Community Forum. A question about ram-high. <https://forum.mikrotik.com/t/a-question-about-ram-high/168910>, 2023. Dowiedzieliśmy się stąd o ram-high które może być przydatne w kontekście naszego projektu.
- [15] Nmap. Nmap reference guide. <https://nmap.org/docs.html>, 2024.
- [16] Nmap. Nmap scripting engine documentation. <https://nmap.org/book/nse.html>, 2024.
- [17] Linux Containers Project. Lxc documentation. <https://linuxcontainers.org/lxc/introduction/>, 2024.
- [18] Linux Containers Project. Lxc vs docker. <https://linuxcontainers.org/lxc/introduction/#containers-vs-docker>, 2024.